

Week 2 – Introduction to Python and Installation Guide

Setting up the working environment

An Introduction to Python

Why Python?

Python is currently the most popular programming language for research and development in Machine Learning. According to Google Trends, the interest in Python for Machine Learning has spiked to an all-new high with other ML languages such as R, Java, Scala, Julia, etc. lagging far behind.

The main qualities of Python are its stability, flexibility, and the availability of a huge variety of tools. Python is:

- **Simple** to understand and **consistent**: this is surely one of the main reasons why it is so popular for Machine Learning. It is simple with an easily readable syntax and that makes it well-loved by both seasoned developers and experimental students. Python's simplicity allows developers to write reliable systems. Developers get to put all their effort into solving an ML problem instead of focusing on the technical nuances of the language.
- **Extremely efficient**: it allows developers to complete more work using fewer lines of code. Many programmers point out the frameworks, libraries, and extensions that simplify the implementation of different functionalities. It is broadly accepted that Python is suitable for collaborative implementation when multiple developers are involved. As a general-purpose language, Python can do a set of complex machine learning tasks and enable you to build prototypes quickly that allows you to test your product for machine learning purposes.
- Python has multiple **Libraries** and **Frameworks**: the implementation of AI and ML algorithms can be tricky and requires a lot of time, thus it becomes crucial to have a well-structured and well-tested environment to enable developers to come up with the best coding solutions. Despite the inherent complexity of some ML algorithms, Python is the rescue by providing developers with an extensive range of libraries and frameworks (a software library is a pre-written code that is used by developers during a common programming task). These libraries help the programmer and reduce development time. Python has a rich technology stack and has a different set of libraries for Machine learning: **Scikit-learn** for machine learning (and Keras and TensorFlow); **SciPy** for advanced computing; **Matplotlib** (and Seaborn) for data visualization; **Pandas** for general-purpose data analysis; **NumPy** for scientific computing and data analysis. Specifically, one of the libraries we will use mostly in this module is **Scikit-learn** which features various classification, regression, and clustering algorithms, including support

vector machines, random forests, gradient boosting, k-means, and DBSCAN, and is designed to work with the Python numerical and scientific libraries NumPy and SciPy. With these solutions, you can develop your product faster. Also, the developers can use the existing libraries to implement necessary features.

- Python is **Flexible**: it is considered the most flexible language in machine learning. It provides various options for users and this flexibility reduces the possibility of errors. It let the programmers take the situation completely under control, and work on it comfortably. Programmers can combine Python with other languages to get the desired results. You do not require recompiling source code. Programmers can implement their changes in coding and see the results quickly.
- Python is **platform independent**: platform independence is surely one of the key aspects of Python's popularity and refers to a programming language or framework allowing developers to implement things on one machine and use them on another machine without any (or with only minimal) changes. Python is supported by many platforms including Linux, Windows, and macOS. Python code can be used to create standalone executable programs for most common operating systems, which means that Python software can be easily distributed and used on those operating systems without a Python interpreter. In addition, developers usually use services such as Google or Amazon for their computing needs. However, you can often find companies and data scientists who use their machines with powerful Graphics Processing Units (GPUs) to train their ML models. And the fact that Python is platform-independent makes this training a lot cheaper and easier.
- Python has **great community** and **Corporate support**: Python has a great supportive community. Because of this support and the simplicity of this language, Python learners can easily improve their Machine Learning knowledge and the implementation, which only leads to increasing popularity. Also, Corporate support is a very important part of the success of Python for ML, as many top companies, such as Google, Facebook, Instagram, Netflix, Quora, etc., use Python for their products. Google is single-handedly responsible for creating many of the Python libraries for Machine Learning such as Keras, TensorFlow, etc. Python AI community has grown across the globe. There are Python forums and an active exchange of experience related to machine learning solutions. For any task you may have, the chance is pretty high that someone else out there has dealt with the same problem. You can find advice and guidance from developers. You will not be alone and are sure to find the best solution to your specific needs if you turn to the Python community.

So, these are some of the reasons you can choose Python for Machine Learning.

Python packages

For scientific computing and computational modelling, and to work with data analysis and Machine Learning, we need some additional libraries (so called **packages**) that are not part of the Python

standard library. These allow us, for example, to create plots, operate on matrices, and use specialised numerical methods. The packages we generally need are

- **numpy** (NUMeric Python): matrices and linear algebra
- **pandas**: Python data science tools (Series and Dataframes)
- **scipy** (SCientific Python): many numerical routines
- **matplotlib**: (PLOTting LIBrary) creating plots of data

The packages numpy, scipy, pandas and matplotlib are building stones of computational work with Python and widely used.

Spyder

Spyder (<https://www.spyder-ide.org/>) is a powerful interactive development environment for the Python language with advanced editing, interactive testing, debugging and introspection features. The name SPYDER derives from "Scientific PYthon Development EnviRonment" (SPYDER).

We will use it as the main environment to learn about Python, programming and computational science and engineering.

Useful features include:

- provision of the **IPython** (Qt) console as an interactive prompt, which can display plots inline
- ability to execute snippets of code from the editor in the console
- continuous parsing of files in editor, and provision of visual warnings about potential errors
- step-by-step execution
- variable explorer

Anaconda

Anaconda (<https://www.anaconda.com/>) is one of several Python distributions. Python distributions provide the Python interpreter, together with a list of Python packages and sometimes other related tools, such as editors. More specifically, Anaconda is not limited to packaging Python packages, but initially emerged to cater for Python-based applications and packages. The packages provided by the Anaconda Python distribution include all of those that we need, and for that reason we suggest using Anaconda in this module. A key part of the Anaconda Python distribution is Spyder, an interactive development environment for Python, including an editor.

Installation

Before you download anything, check if you have Python already installed on your machine (which is likely if you have a Macbook or a Linux machine) by typing the following command in a command shell:

```
python3 -v
```

If you are working on Windows 10 (as in my case), type:

```
python3 --version
```

The output is:

```
Python 3.8.5
```

In general, the installation of the Python interpreter (from source/binaries) is fairly straightforward, but installation of additional packages can be a bit tedious. Instead of doing this manually, we suggest on this page to install the Anaconda Python distribution using these installation instructions, which provides the Python interpreter itself and all packages we need.

The Anaconda Python distribution is available for download for Windows, OS X and Linux operating systems (and free).

For Windows and OS X you are given a choice whether to download the graphical installer or the next based installer. If you don't know what the terminal (OS X) or command prompt (Windows) is, then you are better advised to choose the graphical version. You want to install the default suggestion (Python 3.8, 64bit), not Python 2.

Please go to: <https://www.anaconda.com/products/individual> , download the installer, start it, and follow instructions. Accept default values as suggested.

During the installation, you have the option to install Microsoft Visual Studio Code. You do not need to install the Visual Studio Code environment for this course, but it shouldn't do any harm either.

If you are using Linux and you are happy to use the package manager of your distribution, then you may be better advised to install the required packages individually rather than installing the whole Anaconda distribution.

Here you can find some video tutorials for installation of Python, Anaconda, Spyder and all the main packages:

<https://www.youtube.com/watch?v=UvcQIPZ8ecA> (Python)

<https://www.youtube.com/watch?v=eGVvoIB2I7I> (Anaconda, Python, Jupyter Notebook and Spyder on Windows 10)

<https://www.youtube.com/watch?v=5mDYijMfSzs&t=436s> (How to Install sklearn, numpy, & scipy with Anaconda on Windows 10 64-bit)

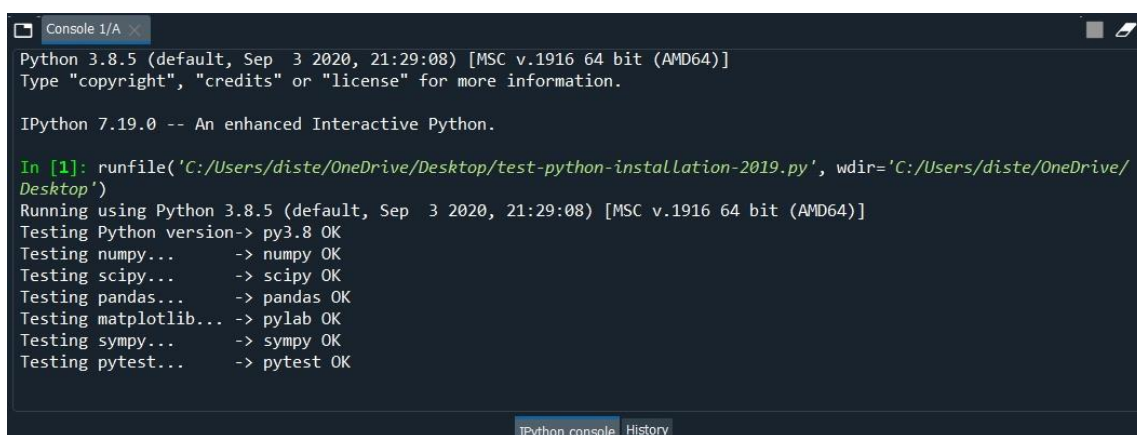
<https://www.youtube.com/watch?v=g6ln1dAt-RI&t=1s> (How to Install Anaconda Python on Windows 10 | 2021 | Download & Install Anaconda | Python)

<https://www.youtube.com/watch?v=C4OPn58BLaU> (How to Set Up Your Data Science Environment)

<https://www.youtube.com/watch?v=kbadAWENkJO> (Download and Install Anaconda Python, Jupyter Notebook and Spyder on Windows 10).

Once finished the installation, you can test the installation by downloading the testing programme included in the Learning material of this week, named as **test-python-installation-2021.py**, and execute it.

To run the test with Spyder, start Spyder (this can be done either by typing Spyder in a terminal or inside the Anaconda Prompt, or by starting Spyder through the Anaconda Navigator), download the testing file, open the file in Spyder via File -> Open, and execute the file via Run -> Run. If you get a pop-up window, you can accept the default settings and click on the run button. You should see output similar to this in the lower right window of Spyder (IPython Console):

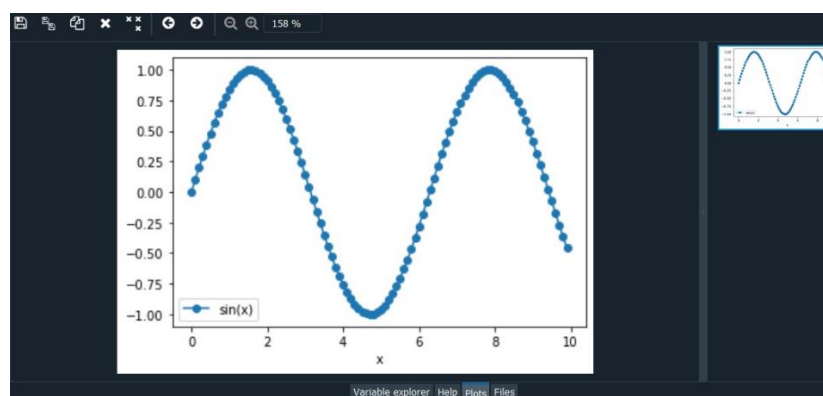


```
Python 3.8.5 (default, Sep 3 2020, 21:29:08) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.19.0 -- An enhanced Interactive Python.

In [1]: runfile('C:/Users/diste/OneDrive/Desktop/test-python-installation-2019.py', wdir='C:/Users/diste/OneDrive/Desktop')
Running using Python 3.8.5 (default, Sep 3 2020, 21:29:08) [MSC v.1916 64 bit (AMD64)]
Testing Python version-> py3.8 OK
Testing numpy... -> numpy OK
Testing scipy... -> scipy OK
Testing pandas... -> pandas OK
Testing matplotlib... -> pylab OK
Testing sympy... -> sympy OK
Testing pytest... -> pytest OK
```

If the test program produces these outputs, it means that Python and the six listed packages are installed correctly. And you should also see a plot appearing, if you go to plot in the upper right window of Spyder:



You can also run the test from the console. In this case, in Windows type cmd in the search box, in Mac OS X, start the Terminal application that is located in the Utilities folder in Applications and in Linux, start one of the shells you have available, or an xterm or so. Download the test file to your machine, change directory into the folder you have downloaded the file to (in my case, C:\Users\diste\OneDrive\Desktop) and type: python test-python-installation-2021.py.

If all the tests pass, you should see output similar to this:

```
C:\Users\diste\OneDrive\Desktop>python test-python-installation-2021.py
Running using Python 3.8.5 (default, Sep  3 2020, 21:29:08) [MSC v.1916 64 bit (AMD64)]
Testing Python version-> py3.8 OK
Testing numpy...      -> numpy OK
Testing scipy...      -> scipy OK
Testing pandas...     -> pandas OK
Testing matplotlib... -> pylab OK
Testing sympy...      -> sympy OK
Testing pytest...     -> pytest OK
```

If you need to update packages in the Anaconda installation, you need to open a terminal (anaconda Powershell prompt) and update the conda program (this manages the updating) by typing the following command into the console: **conda update conda**

Confirm updates if asked to do so. More than one package may be listed to be updated.

The, update individual packages, e.g. Spyder: **conda update spyder**

Other packages you may want to update include ipython, ipython-qtconsole and ipython-notebook. The relevant command would be: **conda update ipython ipython-qtconsole ipython-notebook**

Jupyter

The following link contains a brief introduction to Jupyter, which is a flask-based Python application that enables you to execute Python code in a browser. Instead of Python scripts, you can use Jupyter “notebooks,” which support various interactive features for executing Python code. You get Jupyter Notebook already by downloading Anaconda. You will start / stop the Jupyter Notebook server from the command line: **jupyter notebook**, and to stop the Jupyter Notebook server, just type: **Ctrl+C**. In addition, your knowledge of Jupyter will be very useful when you decide to use Google Colaboratory that also supports Jupyter notebooks in a browser. As I said earlier, you get Jupyter Notebook already by downloading Anaconda. The following are useful videos to understand how to work with Jupyter.

<https://www.youtube.com/watch?v=jZ952vChhul&t=112s>

https://www.youtube.com/watch?v=CwFq3YDU6_Y

https://www.youtube.com/watch?v=2eCHD6f_phE
